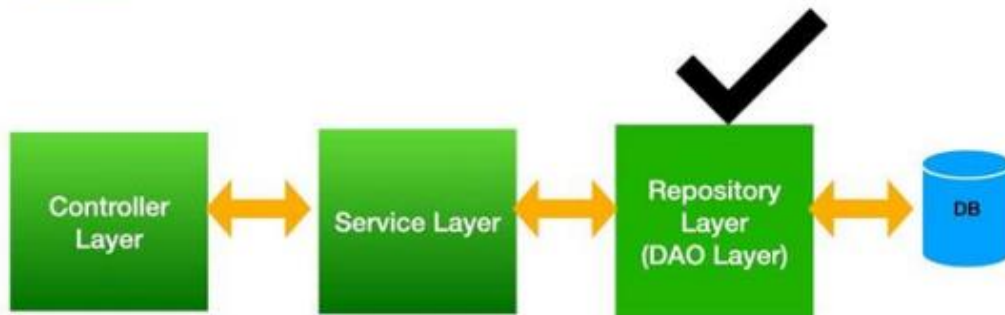


Spring Data JPA



Spring Data JPA



Spring Data JPA 는 Spring Framework에서 JPA를 편리하게 사용할 수 있도록 지원하는 프로젝트로서 CRUD 처리를 위한 공통 인터페이스를 제공한다.

Repository 개발 시 인터페이스만 작성해도 실행 시점에 Spring Data JPA가 구현 객체를 동적으로 생성해서 주입시키므로 데이터 접근 계층을 개발할 때 구현 클래스 없이 인터페이스만 작성해도 개발을 완료할 수 있도록 지원한다.

Spring Data JPA를 사용하기 위해 일반적으로 'JpaRepository' 인터페이스를 상속한 Repository 인터페이스를 정의한다. 단지 인터페이스를 상속했을 뿐인데, 기본적인 메서드를 이미 장착한 상태이고, 심지어 정의한 인터페이스를 구현할 필요도 없다. **Spring이 알아서 해주게 된다.**

Spring Data JPA는

- 지루하게 반복되는 CRUD 문제를 세련된 방법으로 해결
- 개발자는 인터페이스만 작성
- 스프링 데이터 JPA가 구현 객체를 동적으로 생성해서 주입

Spring Data JPA를 사용하지 않았을 경우

```

public class MemberRepository {

    @PersistenceContext
    EntityManager em;

    public void save(Member member) {...}
    public Member findOne(Long id) {...}
    public List<Member> findAll() {...}

    public Member findByUsername(String username) {...}
}

public class ItemRepository {

    @PersistenceContext
    EntityManager em;

    public void save(Item item) {...}
    public Member findOne(Long id) {...}
    public List<Member> findAll() {...}
}

```

→ 반복되는
CRUD

Spring Data JPA를 적용 했을 경우

```

public interface MemberRepository extends JpaRepository<Member, Long> {
    Member findByUsername(String username);
}

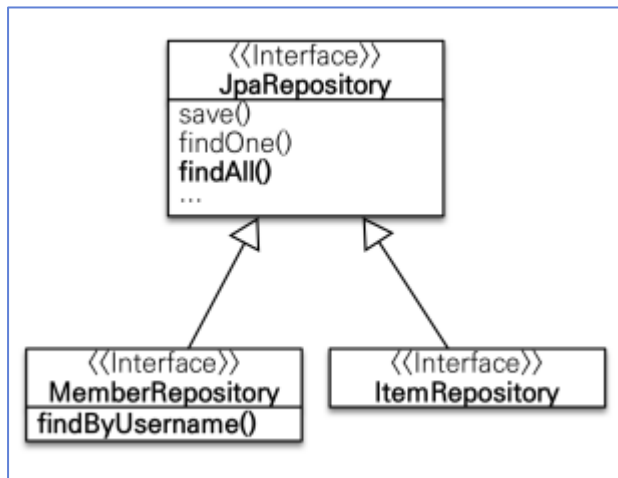
public interface ItemRepository extends JpaRepository<Item, Long> {
}

```

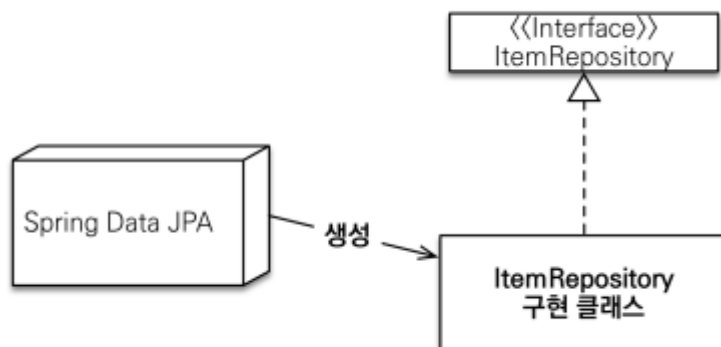
Entity

pk타입

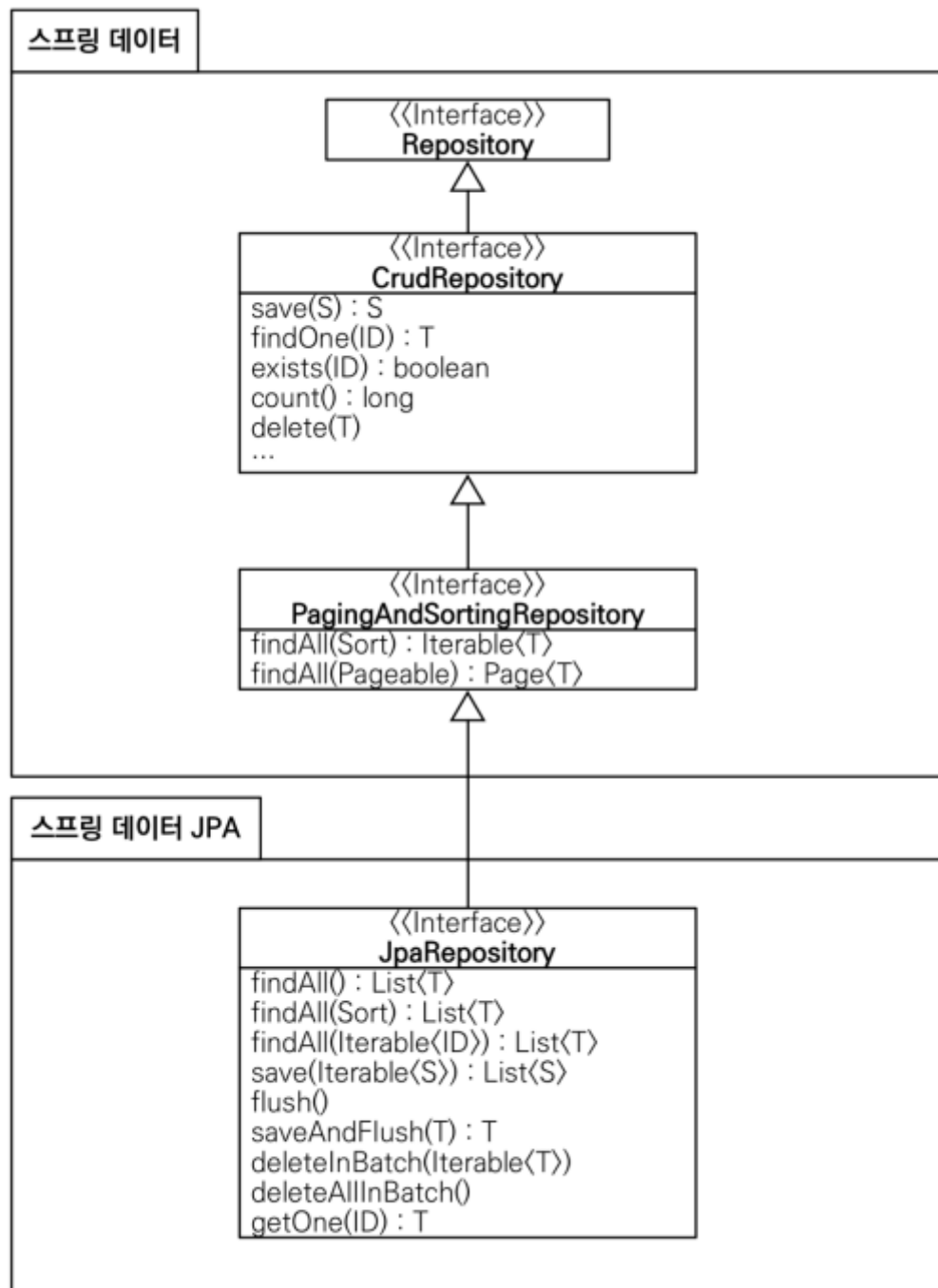
Spring Data JPA를 적용 후 클래스 다이어그램



Spring Data JPA가 구현 클래스 생성



- JpaRepository 인터페이스로 CRUD 기능을 공통으로 처리
- 제네릭으로 설정
- `public interface MemberRepository extends JpaRepository<Member, Long>{ }`



Spring Data JPA는 interface를 만들어서 JpaRepository를 상속 받으면 Spring 내부에서 구현체를 생성해서 주입해준다. JpaRepository는 save(), findAll(), findById(), delete() 등의 기본 CRUD에 충실하게 메소드를 제공한다.

또한, 페이징과 정렬을 쉽게 할 수 있다.

org.springframework.data.domain.Sort: 정렬 기능

org.springframework.data.domain.Pageable: 페이징 기능(내부에 Sort 포함)

```
//count 쿼리 사용
Page<Member> findByName(String name, Pageable pageable);

//count 쿼리 사용 안함
List<Member> findByName(String name, Pageable pageable);

List<Member> findByName(String name, Sort sort);
```

```
public interface Page<T> extends Iterable<T> {

    int getNumber();           //현재 페이지
    int getSize();             //페이지 크기
    int getTotalPages();        //전체 페이지 수
    int getNumberOfElements(); //현재 페이지에 나올 데이터 수
    long getTotalElements();    //전체 데이터 수
    boolean hasPreviousPage();  //이전 페이지 여부
    boolean isFirstPage();      //현재 페이지가 첫 페이지 인지 여부
    boolean hasNextPage();      //다음 페이지 여부
    boolean isLastPage();       //현재 페이지가 마지막 페이지 인지 여부
    Pageable nextPageable();    //다음 페이지 객체, 다음 페이지가 없으면 null
    Pageable previousPageable(); //다음 페이지 객체, 이전 페이지가 없으면 null
    List<T> getContent();        //조회된 데이터
    boolean hasContent();        //조회된 데이터 존재 여부
    Sort getSort();             //정렬 정보
}
```

하지만,

위의 메서드들은 모든 엔티티에 대해 공통으로 쓰일 수 있는 메소드를 제공한다. 사실 비즈니스 로직을 다루는 것은 그리 간단하지 않다. 조건을 지정하여 조회하거나, 제거하거나 저장할 수 있는 기능들을 커스텀마이징 해야 한다.

그래서, 아래의 문법을 제공한다.

- 1) Query메소드 : 메소드이름 쿼리 생성
- 2) JPQL문법 : @Query안에 객체중심으로 쿼리를 작성
- 3) QueryDSL : JPQL을 자동으로 생성

📖 Query메소드

메서드 이름으로 쿼리 생성

<https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>

쿼리 종류	이름 규칙
조회	<code>find...By</code> , <code>read...By</code> , <code>query...By</code> , <code>get...By</code>
COUNT	<code>count...By</code> 반환타입 long
EXISTS	<code>exists...By</code> 반환타입 boolean
삭제	<code>delete...By</code> , <code>remove...By</code>
DISTINCT	<code>findDistinct</code> , <code>findMemberDistinctBy</code>
LIMIT	<code>findFirst3</code> , <code>findFirst</code> , <code>findTop</code> , <code>findTop3</code>

```

////QueryMethod 작성////////////////////////////////////
/**
 * 전달된 글번호보다 작고 전달된 작성자와 동일한 레코드 검색
 * : findByXxx... 시작하는 메소드 작성
 * : https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#jpa.query-methods.query-creation
 * : https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html
 * */
1 usage
List<Board> findByBnoLessThanAndWriter(Long bno, String writer);

```

📖 @Query를 이용하여 JPQL을 작성해서 쿼리 수행

```

2 usages
11 public interface BoardRepository extends JpaRepository<Board, Long> {
12     /**
13      * 인수로 전달된 글번호가 큰 레코드 삭제하고싶다
14      * : JPQL 문법을 작성할때 @Query 선언
15      * : DML 작업을 할때는 반드시 @Modifying 선언
16      */
17     @Query(value = "delete from Board b where b.bno > ?1 ")
18     @Modifying
19     void deleteGreaterThan(Long bno);
20
21
22     /**
23      * 글번호 or 제목에 해당하는 레코드 검색
24      * nativeQuery = true 옵션은 DB 쿼리 중심으로 쿼리를 작성한다.
25      */
26     // @Query("select b from Board b where b.bno > ?1 or b.title = ?2")
27     @Query(value = "select * from board where bno > ?1 or TITLE = ?2", nativeQuery = true)
28     List<Board> selectByBnoTitle(Long bno, String title);
29
30
31     /**
32      * 글번호, 제목, 작성자에 해당하는 레코드 검색
33      */
34     @Query("select b from Board b where b.bno = :#{bo.bno} or b.title = :#{bo.title} or b.writer = :#{bo.writer}")
35     List<Board> selectByParamsBoard(@Param("bo") Board board);
36

```

Spring Data JPA가 제공하는 JpaRepository 인터페이스를 상속받으면 상속하는 것만으로도 기본적인 CRUD 기능을 사용할 수 있다. 그런데 이름으로 검색하거나, 가격으로 검색하는 기능은 공통으로 제공할 수 있는 기능이 아니다. 따라서 이런 경우는 쿼리 메서드 기능을 사용하거나 @Query를 사용해서 직접 쿼리를 실행한다

```

public interface SpringDataJpaItemRepository extends JpaRepository<Item, Long> {
    List<Item> findByItemNameLike(String itemName);
    List<Item> findByPriceLessThanEqual(Integer price);
    //쿼리 메서드 (아래 메서드와 같은 기능 수행)
    List<Item> findByItemNameLikeAndPriceLessThanEqual(
        String itemName, Integer price);

    //쿼리 직접 실행
    @Query(
        "select i from Item i where i.itemName like :itemName and i.price <= :price")
    List<Item> findItems(@Param("itemName") String itemName,
        @Param("price") Integer price);
}

```

위의 코드 데이터를 조건에 따라 4가지로 분류해서 검색한다.

1) 모든 데이터 조회

- findAll()

코드에는 보이지 않지만 JpaRepository 공통 인터페이스가 제공하는 기능이다.
모든 Item 을 조회한다.

실행되는 JPQL - select i from Item i

2) 이름 조회

- findByItemNameLike()

이름 조건만 검색했을 때 사용하는 쿼리 메서드이다.

실행되는 JPQL - select i from Item i where i.name like ?

3) 가격 조회

- findByPriceLessThanEqual()

가격 조건만 검색했을 때 사용하는 쿼리 메서드이다.

실행되는 JPQL - `select i from Item i where i.price <= ?`

4) 이름 + 가격 조회

- findByItemNameLikeAndPriceLessThanEqual()

이름과 가격 조건을 검색했을 때 사용하는 쿼리 메서드이다.

실행되는 JPQL - `select i from Item i where i.itemName like ? and i.price <= ?`

5) 쿼리 직접 작성

- findItems()

메서드 이름으로 쿼리를 실행하는 기능은 다음과 같은 단점이 있다.

1. 조건이 많으면 메서드 이름이 너무 길어진다.
2. 조인 같은 복잡한 조건을 사용할 수 없다.

메서드 이름으로 쿼리를 실행하는 기능은 간단한 경우에는 매우 유용하지만, 복잡해지면 직접 JPQL 쿼리를 작성하는 것이 좋다. 쿼리를 직접 실행하려면 @Query 어노테이션을 사용한다.

메서드 이름으로 쿼리를 실행할 때는 파라미터를 순서대로 입력하면 되지만, 쿼리를 직접 실행할 때는 파라미터를 명시적으로 바인딩 한다.

파라미터 바인딩은

@Param("itemName") 어노테이션을 사용하고, 어노테이션의 값에 파라미터 이름을 입력한다.

쿼리 메서드를 이용하는 방법과 @Query로 JPQL을 사용하는 방법 모두 복잡한 동적 쿼리에는 좀 부족한 편이다.

복잡한 동적 쿼리는 QueryDSL을 사용하는 것이 좋다.

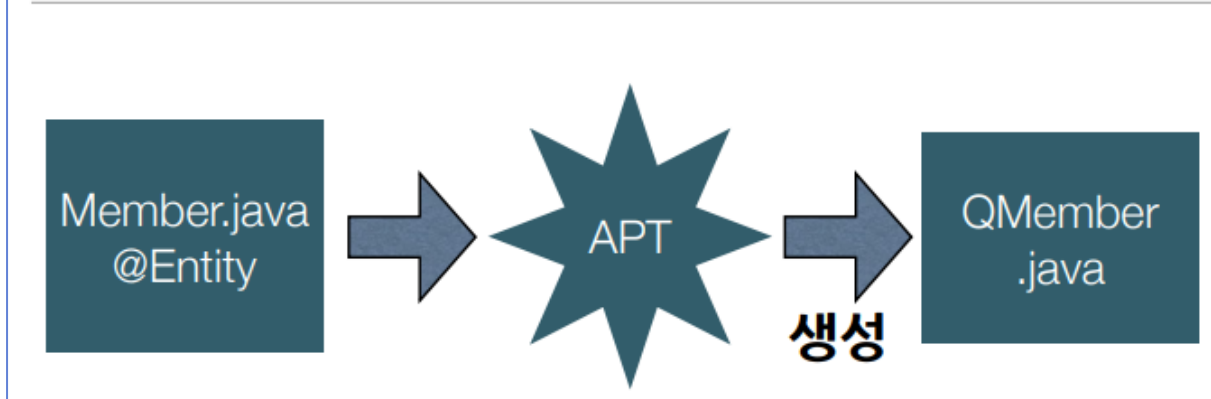
QueryDSL은 JPQL을 코드로 작성할 수 있도록 도와주는 빌더 API이다.

QueryDSL 장점

- 문자가 아닌 코드로 작성
- 컴파일 시점에 오류 발견
- 코드 자동완성
- 단순함, 쉬움: 코드 모양이 JPQL과 거의 흡사.
- 동적 쿼리



QueryDSL - 쿼리타입 생성



- QueryDSL사용

```
JPAQuery query = new JPAQuery(em);  
  
QMember m = QMember.member;  
List<Member> list =  
    query.from(m)  
        .where(m.age.gt(18))  
        .list(m);
```

QueryDSL조인

```
JPAQuery query = new JPAQuery(em);  
  
QMember m = QMember.member;  
QTeam t = QTeam.team;  
  
List<Member> list = query.from(m)  
    .join(m.team, t)  
    .where(t.name.eq("teamA"))  
    .list(m);
```

@QueryDSL – 페이징 API

```
JPAQuery query = new JPAQuery(em);  
QMember m = QMember.member;  
  
List<Member> list = query.from(m)  
    .orderBy(m.age.desc())  
    .offset(10)  
    .limit(20)  
    .list(m);
```

QueryDSL – 동적쿼리

```
String name = "member";
int age = 9;

JPAQuery query = new JPAQuery(em);
QMember m = QMember.member;

BooleanBuilder builder = new BooleanBuilder();
if (name != null) {
    builder.and(m.name.contains(name));
}
if (age != 0) {
    builder.and(m.age.gt(age));
}

List<Member> list = query.from(m)
    .where(builder)
    .list(m);
```

[객체와 객체 의 관계]

PK FK HUMAN_ID HUMAN_NAME ANIMAL_ID	PK ANIMAL_ID ANIMAL_NAME
--	---


```
@Entity
@Table(name = "human")
public class Human {
    @Id
    @GeneratedValue
    @Column(name = "human_id")
    private long id;

    @Column(name = "human_name")
    private String name;

    @ManyToOne
    @JoinColumn(name = "animal_id")
    private Animal animal;
}
```

```
@Entity
@Table(name = "animal")
public class Animal {
    @Id
    @GeneratedValue
    @Column(name = "animal_id")
    private long id;

    @Column(name = "animal_name")
    private String name;
}
```

[Spring Boot Application 에 설정된 어노테이션 설명]

```
@SpringBootApplication
@EnableJpaRepositories(basePackages = {"com.example.springedu.repository"})
@EntityScan(basePackages = {" com.example.springedu.entity"})
```

- @SpringBootApplication 어노테이션은 스프링 부트의 가장 기본적인 설정을 선언한다. 내부적으로 @ComponentScan과 @EnableAutoConfiguration을 설정한다.
- @ComponentScan 어노테이션은 스프링 3.1부터 도입된 어노테이션이며 스캔 위치를 설정한다. (프로젝트 생성시 기본으로 만들어지는 패키지의 하위에 클래스들을 두는 경우에는 생략 가능)
- @EnableJpaRepositories 어노테이션은 JPA Repository들을 활성화하기 위한 어노테이션이다.
- @EntityScan 어노테이션은 엔티티 클래스를 스캔할 곳을 지정하는데 사용한다.

[Spring Boot 테스트]

스프링 부트는 서블릿 기반의 웹 개발을 위한 spring-boot-starter-web, 유효성 검증을 위한 spring-boot-starter-validation 등 spring-boot-starter 의존성을 제공하고 있다. 테스트를 위한 spring-boot-starter-test 역시 존재하는데, 다음과 같은 라이브러리들이 포함된다.

- 1) JUnit 5: 자바 애플리케이션의 단위 테스트를 위한 사실상의 표준 테스트 도구
- 2) Spring Test & Spring Boot Test: 스프링 부트 애플리케이션에 대한 유틸리티 및 통합 테스트 지원
- 3) AssertJ: 유연한 검증 라이브러리
- 4) Hamcrest: 객체 Matcher를 위한 라이브러리
- 5) Mockito: 자바 모킹 프레임워크
- 6) JSONassert: JSON 검증을 위한 도구
- 7) JsonPath: JSON용 XPath

spring-boot-starter-test 의존성을 추가하면 스프링이 미리 만들어둔 테스트를 위한 다양한 어노테이션을 사용해 편리하게 테스트를 작성할 수 있다.

```
@SpringBootTest
@WebMvcTest
@DataJpaTest
@RestClientTest
@JsonTest
@JdbcTest
.....
[ @DataJpaTest 애노테이션 ]

@DataJpaTest
```

JPA 레포지토리 테스트를 위해서는 @DataJpaTest를 이용할 수 있다.

@DataJpaTest는 기본적으로 @Entity가 있는 엔티티 클래스들을 스캔하며 테스트를 위한 TestEntityManager를 사용해 JPA 레포지토리들을 설정해준다.

스프링은 테스트에 @Transactional이 있으면 테스트가 끝난 후 자동으로 트랜잭션을 롤백한다.

@DataJpaTest에는 @Transactional 어노테이션이 포함되어 있어서 기본적으로 모든 테스트가 롤백된다. 만약 롤백을 원하지 않는다면 @Rollback(false)를 추가하면 된다.

```
@DataJpaTest
@Rollback(false)
class MyRepositoryTests {

    // ...

}
```

또한 만약 H2와 같은 내장 데이터베이스가 클래스 패스에 존재한다면 내장 데이터베이스가 자동

구성된다. spring-boot-test 의존성에는 기본적으로 H2가 들어있으므로 별다른 설정을 주지 않는다면 H2로 설정된다. 내장 데이터베이스로 설정되기를 원하지 않는다면 다음과 같이 AutoConfigureTestDatabase의 replace 속성을 NONE으로 주면 된다.

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = Replace.NONE)
class MyRepositoryTests {

    // ...

}
```

테스트 어노테이션	@SpringBootTest	@WebMvcTest	@DataJpaTest
테스트 종류	통합테스트	부분테스트 (슬라이스 테스트)	부분테스트 (슬라이스 테스트)
등록되는 빈	모든 빈	Controller와 연관된 빈	JPA Repository와 연관된 빈